How-To Library

Informatica

Configuring Kerberos-enabled Kafka for a Big Data Streaming Mapping in 10.2.2

© Copyright Informatica LLC 2019, 2021. Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html. Other company and product names may be trade names or trademarks of their respective owners.

Abstract

You can use Kerberos-enabled Kafka as a source or target in a streaming mapping. This article describes how to configure Kerberos-enabled Kafka for a streaming mapping in Big Data Streaming 10.2.2.

Supported Versions

- Big Data Streaming 10.2.2

Table of Contents

Overview. 2
Prerequisites. 2
Configure Default Realm and KDC. 2
Configure Java Authorization and Authentication Service. 4
Create a Kerberos-enabled Kafka Connection. 6
Order of Precedence for the Connection Properties. 7

Overview

Apache Kafka is a publish-subscribe messaging system that manages and maintains the real-time stream of data. With Kafka, any user or application can write any messages to any topic or read data from any topic.

To communicate securely with Kafka, you can use Kerberos authentication. Kerberos is a network authentication protocol that uses tickets to authenticate access to services and nodes in a network.

Kerberos uses a Key Distribution Center (KDC) to validate the identities of users and services and to grant tickets to the authenticated users and service accounts.

Kerberos uses keytab files to authenticate Informatica clients with services that run within the domain. A keytab file contains the service principal name (SPN) and associated encrypted keys. SPN is a unique identifier of a service instance.

Prerequisites

Before you create a Kerberos-enabled Kafka connection, perform the following tasks:

1. Configure the default realm and KDC.
2. Configure the Java Authorization and Authentication Service (JAAS).

Configure Default Realm and KDC

Default realm is the name of the Kerberos realm to which the Informatica domain services belong. The realm name must be in uppercase.

Kerberos stores configuration information in a file named `krb5.conf`. The configuration file contains the information about the Kerberos server, including the Kerberos realm and the address of the KDC.

Before you read from or write to a Kerberos-enabled Kafka, perform the following tasks on all cluster nodes, domain nodes, and the Developer tool machine:

1. Configure the cluster and domain nodes.
2. Configure the Developer tool.

Step 1. Configure the cluster and domain nodes

Use the `krb5.conf` file to configure the cluster and domain nodes.

1. Ensure that the `krb5.conf` file is located in the `/etc` directory.
2. Ensure that the `krb5.conf` contains the following configuration:

```
[libdefaults]
default_realm=<REALM NAME>
dns_lookup_realm=false
dns_lookup_kdc=false
ticket_lifetime=24h
renew_lifetime=7d
forwardable=true

[realms]
<REALM NAME>={
kdc=<location where KDC is installed>
admin_server=<location where KDC is installed> }

[domain_realm]
<domain name or host name>=<KERBEROS DOMAIN NAME>
<domain name or host name>=<KERBEROS DOMAIN NAME>
```

3. Ensure that the `/etc/hosts` file has an entry for the Key Distribution Center (KDC) server IP address, fully qualified domain name (FQDN), and KDC alias.
   You can use the following host name format in lowercase:

   ```
   <kdc server ip address>  <kdc server fqdn>  <kdc alias>
   ```

   For example: `10.55.121.8 mykdc.informatica.com mykdc`
4. To verify if the KDC server is available, ping the FQDN of the KDC server by using the following command:

   ```
   ping <KDC FQDN>
   ```

   For example: `ping mykdc.informatica.com`

Step 2. Configure the Developer tool

Use the `krb5.conf` file to configure the Developer tool.

1. Rename the Kerberos configuration file from `krb5.conf` to `krb5.ini`.
2. Ensure that the `krb5.ini` file is located in the following location on Windows:

   ```
   C:\Windows
   ```
3. Ensure that the host file located in the following location has an entry for the KDC server IP address, fully qualified domain name, and KDC alias:

   ```
   C:\Windows\System32\drivers\etc\hosts
   ```

   You can use the following host name format in lowercase:

   ```
   <kdc server ip address>  <kdc server fqdn>  <kdc alias>
   ```
4. To verify if the KDC server is available, ping the FQDN of the KDC server by using the following command:

   ```
   ping <KDC FQDN>
   ```

   For example: `ping mykdc.informatica.com`

Configure Java Authorization and Authentication Service

The Java Authorization and Authentication Service (JAAS) login configuration file contains entries that specify the authentication technologies for the applications. Any client that connects to a Kerberos-enabled cluster uses the settings in the JAAS login configuration file.

To pass static JAAS configuration file into the JVM, perform the following tasks:

1. Create a static JAAS configuration file.
2. Embed the JAAS configuration.

Step 1. Create a static JAAS configuration file

Use the `java.security.auth.login.config` property at run time.

1. Create a JAAS configuration file and update the keytab file details.
   For more information on how to create JAAS configuration and configure keytab for Kafka clients, see the Apache Kafka documentation at https://kafka.apache.org/0101/documentation/#security.

   For example, the JAAS configuration file can conatin the following configuration:

   ```
   //Kafka client authentication. Used for client to Kafka broker connection.
   KafkaClient{
   com.sun.security.auth.module.Krb5LoginModule required
   doNotPrompt=true
   useKeyTab=true
   storeKey=true
   keyTab="<keytab file path>/<keytab file name>'
   principal="<principal name>"
   client=true
   };
   ```

2. On all the nodes of the Hadoop cluster, place the JAAS configuration file and the keytab file in the same location.
   For example, use the following location based on your environment:

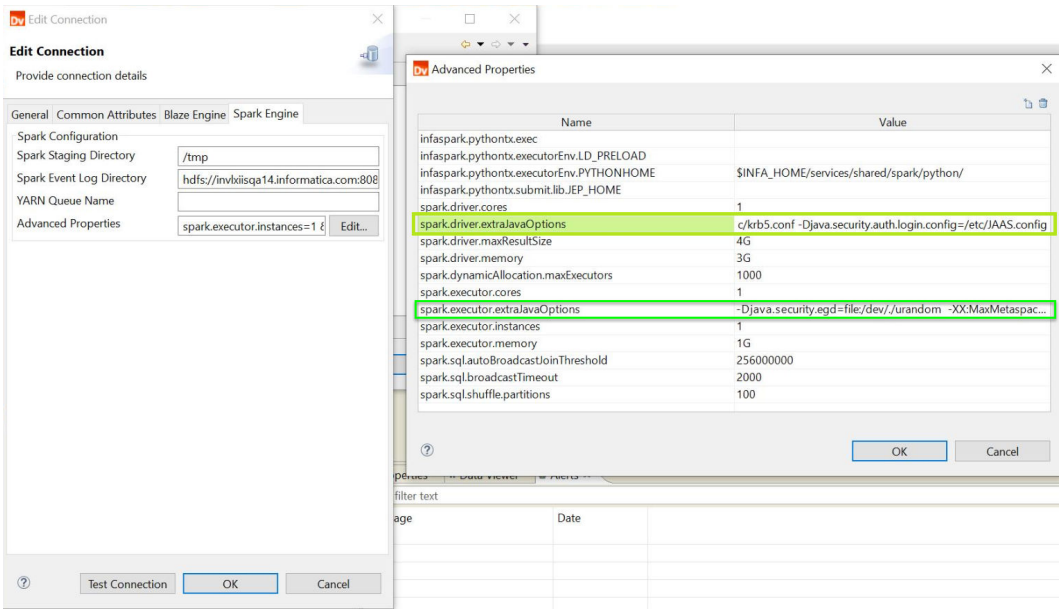
   - On UNIX. `/etc` or `/temp`.
   - On Windows. `C:\Windows`.

3. On the **Spark Engine** tab of the Hadoop connection properties, update the **spark.driver.extraJavaOptions** and **spark.executor.extraJavaOptions** properties in the **Advanced Properties** property in the following format:

   ```
   spark.executor.extraJavaOptions=-Djava.security.egd=file:/dev/./urandom
   -XX:MaxMetaspaceSize=256M -
   Djavax.security.auth.useSubjectCredsOnly=true
   -Djava.security.krb5.conf=/<krb5.conf file path>/krb5.conf
   -Djava.security.auth.login.config=/<JAAS
   config path>/<kafka_client_jaas>.config

   spark.driver.cluster.mode.extraJavaOptions=-
   Djava.security.egd=file:/dev/./urandom
   -XX:MaxMetaspaceSize=256M -
   Djavax.security.auth.useSubjectCredsOnly=true
   -Djava.security.krb5.conf=/<krb5.conf file path>/krb5.conf
   -Djava.security.auth.login.config=<JAAS
   config path>/<kafka_client_jaas>.config
   ```

   The following image shows the **Advanced Properties**:



4. Configure the following properties in the data object read or write operation:
   - Data object read operation. Configure the **Consumer Configuration Properties** property in the advanced properties.
   - Data object write operation. Configure the **Producer Configuration Properties** property in the advanced properties.

   You can configure the following properties:

   ```
   security.protocol=SASL_PLAINTEXT,sasl.kerberos.service.name=kafka,sasl.
   mechanism=GSSAPI
   ```

Step 2. Embed the JAAS configuration

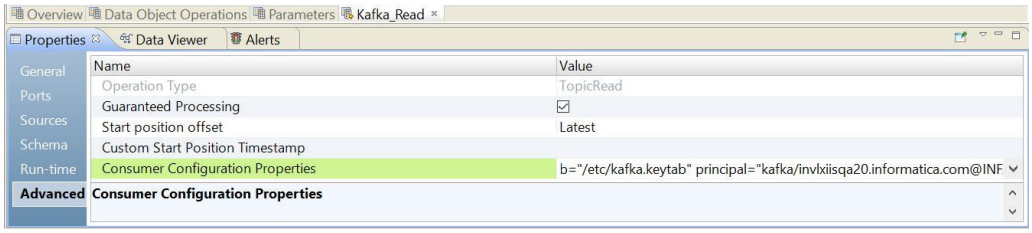
You must embed the JAAS configuration in the **sasl.jaas.config** property.

1. On the **Spark Engine** tab of the Hadoop connection properties, update the **spark.driver.extraJavaOptions** and **spark.executor.extraJavaOptions** properties in the **Advanced Properties** property in the following format:

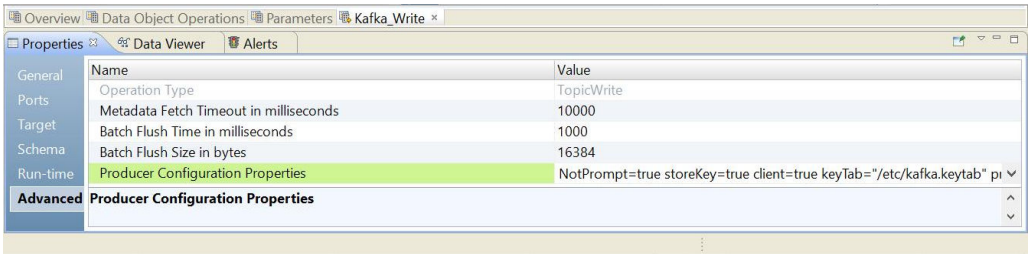
   ```
   spark.executor.extraJavaOptions=-Djava.security.egd=file:/dev/./urandom
   -XX:MaxMetaspaceSize=256M -XX:+UseG1GC -XX:MaxGCPauseMillis=500 -
   Djava.security.krb5.conf=<krb5.conf file path>

   spark.driver.cluster.mode.extraJavaOptions=-
   Djava.security.egd=file:/dev/./urandom
   -XX:MaxMetaspaceSize=256M -XX:+UseG1GC -XX:MaxGCPauseMillis=500 -
   Djava.security.krb5.conf=<krb5.conf file path>
   ```

2. Configure the following properties in the data object read or write operation:
   - Data object read operation. Configure the **Consumer Configuration Properties** property in the advanced properties.



- Data object write operation. Configure the **Producer Configuration Properties** property in the advanced properties.



You can configure the following properties:

```
security.protocol=SASL_PLAINTEXT,sasl.kerberos.service.name=kafka,sasl.
mechanism=GSSAPI,sasl.jaas.config=com.sun.security.auth.module.Krb5Logi
nModule required useKeyTab=true
storeKey=true doNotPrompt=true serviceName="<service_name>"
keyTab="<location of keytab file>"
client=true principal="<principal_name>";
```

Create a Kerberos-enabled Kafka Connection

You can create a Kerberos-enabled Kafka connection in the Developer or Administrator tool.

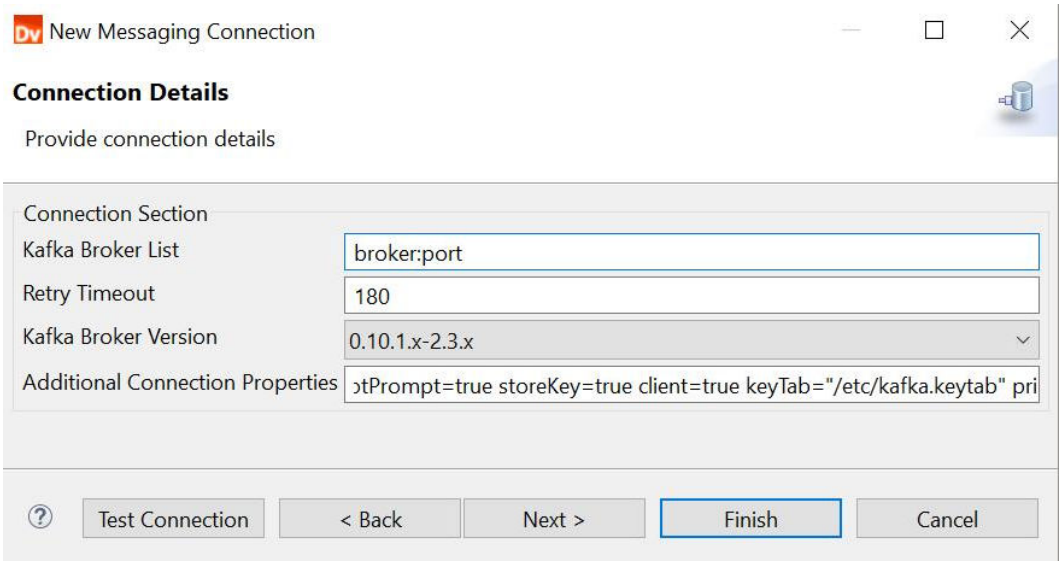
You can place the keytab file in the same directory path on all cluster nodes, domain nodes, and the Developer tool machine.

For example, if a keytab file on Linux named `kafka.keytab` resides at `<Install dir>/services/shared/security/keytabs/kafka.keytab`, place the keytab file on Windows in the same directory path as Linux, such as `C:\<Install dir>\services\shared\security\keytabs\kafka.keytab`.

1. Create a messaging connection. Add the name of the connection and the connection details.
2. Configure the **Additional Connection Properties** property in the following format:

```
security.protocol=SASL_PLAINTEXT,sasl.kerberos.service.name=kafka,sasl.
mechanism=GSSAPI,sasl.jaas.config=com.sun.security.auth.module.Krb5Logi
nModule required useKeyTab=true doNotPrompt=true storeKey=true
client=true keyTab=<keytab location on local machine>
principal=<principal>;
```

The following image shows the connection details for a new messaging connection:



3. Test the connection, and click **Finish**.

Order of Precedence for the Connection Properties

If you configure a connection property in more than one location, the value is overridden based on the following order of precedence:

1. Read or write operation set using the **Consumer Configuration Properties** and **Producer Configuration Properties** properties.
2. Connection details set using the **Additional Connection Properties** property.
3. Advanced properties set using the **spark.driver.extraJavaOptions** and **spark.executor.extraJavaOptions** properties.

Authors

Tanvi Sandeep

Rajat Sharda